API

Description	api instead of web scraping

API, or Application Programming Interfaces, serve as a more structured and formal alternative to web scraping for accessing data online.

Unlike web scraping, which involves direct extraction of data from web pages, APIs allow developers to access structured data and resources of an application or service more efficiently and in a compliant manner.

> 📌 While web scraping can be a valid solution in many situations, especially when official APIs are unavailable or limited, the use of APIs is generally preferred when possible.

Here are some considerations on using APIs as an alternative to web scraping:

1. **Data Structuring:** APIs often provide data in a more structured format compared to web scraping. Data is typically returned in JSON or XML format, making it easier to process and integrate into programs.
2. **Authorized Access:** Many APIs require the use of API keys or access tokens, providing an authentication mechanism and access control. This helps ensure that only authorized users can access the data.
3. **Greater Stability:** APIs are designed to be more stable than web scraping. API endpoints are often versioned, allowing developers to maintain compatibility with older versions while introducing improvements.
4. **Flexibility and Scalability:** APIs often offer a wide range of features, allowing developers to select and retrieve specific sets of data. Additionally, APIs are designed to handle higher workloads compared to web scraping, making them more scalable.

5. **Real-Time Updates:** Some APIs provide real-time data, allowing developers to access the latest information without the need to actively perform web scraping at regular intervals.

Molto spesso le API sono utilizzate con il paradigma REST

Paradigma REST

Nel modello **REST**, le risorse sono considerate entità identificabili, come documenti, immagini, o dati. Ogni risorsa è unica e può essere identificata da un URI (Uniform Resource Identifier).

Le operazioni fondamentali eseguite su risorse sono Create (Creazione), Read (Lettura), Update (Aggiornamento), e Delete (Cancellazione), spesso abbreviate con l'acronimo CRUD.

Queste operazioni corrispondono alle azioni di base eseguite sui dati.

Un'architettura RESTful è stateless, il che significa che ogni richiesta del client al server contiene tutte le informazioni necessarie per comprendere e soddisfare quella richiesta. Il server non mantiene uno stato delle sessioni tra le richieste.

Le risorse possono essere contrassegnate come cacheable o non-cacheable. Questo consente ai client di memorizzare in modo efficiente le risorse e migliorare le prestazioni attraverso l'uso di caching.